

ANSH THAPA

CU24250224

B.TECH(CSE)

Experiment No. 9

Experiment Name

Object Detection and Recognition using Deep Learning Models on Standard Image Datasets

Aim

To implement a deep learning-based object detection and recognition system using pre-trained models and evaluate its performance on standard image datasets for identifying and localizing multiple objects.

Purpose

Object detection is one of the most significant tasks in computer vision, combining object localization and object classification within an image. Unlike image classification, which predicts only the image category, object detection identifies the location of one or more objects by generating bounding boxes along with class labels.

In this experiment, students will utilize pre-trained deep learning models such as YOLOv8, SSD (Single Shot Detector), or Faster R-CNN to detect and recognize objects from standard datasets such as MS COCO, Pascal VOC, or Open Images Dataset. The experiment provides practical exposure to modern object detection frameworks widely used in autonomous vehicles, surveillance systems, healthcare, and smart city applications.

Prerequisites

- Basic knowledge of Digital Image Processing
- Familiarity with Python programming
- Python libraries: OpenCV (cv2), Ultralytics YOLO, PyTorch/TensorFlow, NumPy, and

Matplotlib

- Understanding of Convolutional Neural Networks (CNNs)
- Jupyter Notebook, Google Colab, or Visual Studio Code

In [2]:

```
# Step 1: Install required libraries
!pip install ultralytics opencv-python matplotlib

# Import libraries
import cv2
import matplotlib.pyplot as plt
from ultralytics import YOLO
import time

# Step 2: Load pre-trained YOLOv8 model
model = YOLO("yolov8n.pt")

# Step 3: Load your uploaded image
image_path = "/content/07met-fares-01-gqhw-articleLarge.avif"
img = cv2.imread(image_path)

# Check whether image was loaded successfully
if img is None:
    print("Error: Image not found!")
else:
    print("Image loaded successfully!")

# Step 4: Image preprocessing
img = cv2.resize(img, (640, 640))

# Display input image
plt.figure(figsize=(8, 6))
```

```

plt.figure(figsize=(8, 8))
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title("Input Image")
plt.axis("off")
plt.show()

# Step 5: Apply Object Detection
start_time = time.time()

results = model(img)

end_time = time.time()

# Calculate inference time
inference_time = end_time - start_time

# Step 6: Display detected objects
result = results[0]

annotated_img = result.plot()

plt.figure(figsize=(12, 8))
plt.imshow(cv2.cvtColor(annotated_img, cv2.COLOR_BGR2RGB))
plt.title("Object Detection Result")
plt.axis("off")
plt.show()

# Step 7: Analyze detection results
print("\nDetected Objects:")
print("-----")

if len(result.bboxes) == 0:
    print("No objects detected.")
else:
    for box in result.bboxes:
        class_id = int(box.cls[0])
        confidence = float(box.conf[0])

        class_name = model.names[class_id]

        print(
            "Object:", class_name,
            "| Confidence:", round(confidence, 2)
        )

# Inference speed
print("\nInference Time:",
      round(inference_time, 3),
      "seconds")

# Step 8: Model performance information
print("\nModel Used: YOLOv8 Nano")
print("Model File: yolov8n.pt")

# Step 9: Applications
print("\nReal-World Applications:")
print("1. Autonomous Driving")
print("2. Surveillance")
print("3. Retail Analytics")
print("4. Industrial Inspection")

# Step 10: Conclusion
print("\nConclusion:")
print("YOLOv8 successfully detects and localizes objects")
print("using bounding boxes, class labels and confidence scores.")
print("The model is fast and suitable for real-time object detection.")

```

Requirement already satisfied: ultralytics in /usr/local/lib/python3.13/dist-packages (8.4.165)

Requirement already satisfied: opencv-python in /usr/local/lib/python3.13/dist-packages (5.0.0.93)

Requirement already satisfied: matplotlib in /usr/local/lib/python3.13/dist-packages (3.10.0)

Requirement already satisfied: cloudpickle>=3.1.1 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (3.1.2)

Requirement already satisfied: filelock>=3.16.1 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (3.32.7)

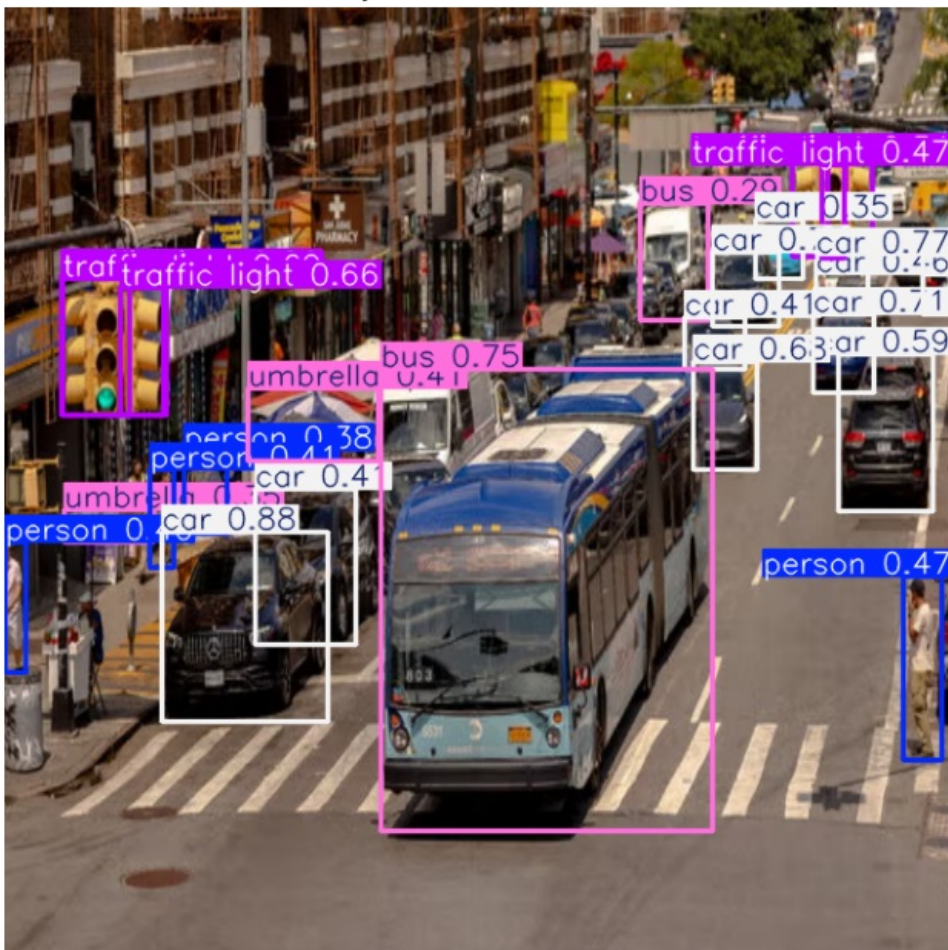
Requirement already satisfied: numpy>=1.23.0 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (2.1.3)
Requirement already satisfied: pillow>=7.1.2 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (11.3.0)
Requirement already satisfied: pyyaml>=5.3.1 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (6.0.3)
Requirement already satisfied: requests>=2.23.0 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (2.32.4)
Requirement already satisfied: torch>=1.8.0 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (2.11.0+cpu)
Requirement already satisfied: torchvision>=0.9.0 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (0.26.0+cpu)
Requirement already satisfied: psutil>=5.8.0 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (5.9.5)
Requirement already satisfied: polars>=0.20.0 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (1.35.2)
Requirement already satisfied: nvidia-ml-py>=12.0.0 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (13.615.71)
Requirement already satisfied: ultralytics-thop>=2.2.0 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (2.2.1)
Requirement already satisfied: ultralytics-platform>=0.1.45 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (0.1.70)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.13/dist-packages (from matplotlib) (1.4.0)
Requirement already satisfied: cycycler>=0.10 in /usr/local/lib/python3.13/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.13/dist-packages (from matplotlib) (4.65.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.13/dist-packages (from matplotlib) (1.5.1)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.13/dist-packages (from matplotlib) (26.3)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.13/dist-packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.13/dist-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: polars-runtime-32==1.35.2 in /usr/local/lib/python3.13/dist-packages (from polars>=0.20.0->ultralytics) (1.35.2)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.13/dist-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.13/dist-packages (from requests>=2.23.0->ultralytics) (3.4.9)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.13/dist-packages (from requests>=2.23.0->ultralytics) (3.19)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.13/dist-packages (from requests>=2.23.0->ultralytics) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.13/dist-packages (from requests>=2.23.0->ultralytics) (2026.7.22)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.13/dist-packages (from torch>=1.8.0->ultralytics) (4.16.0)
Requirement already satisfied: setuptools<82 in /usr/local/lib/python3.13/dist-packages (from torch>=1.8.0->ultralytics) (80.10.2)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.13/dist-packages (from torch>=1.8.0->ultralytics) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.13/dist-packages (from torch>=1.8.0->ultralytics) (3.6.1)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.13/dist-packages (from torch>=1.8.0->ultralytics) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.13/dist-packages (from torch>=1.8.0->ultralytics) (2025.12.0)
Requirement already satisfied: httpx<1,>=0.28 in /usr/local/lib/python3.13/dist-packages (from ultralytics-platform>=0.1.45->ultralytics) (0.28.1)
Requirement already satisfied: anyio in /usr/local/lib/python3.13/dist-packages (from httpx<1,>=0.28->ultralytics-platform>=0.1.45->ultralytics) (4.15.1)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.13/dist-packages (from httpx<1,>=0.28->ultralytics-platform>=0.1.45->ultralytics) (1.0.9)
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.13/dist-packages (from httpcore==1.*->httpx<1,>=0.28->ultralytics-platform>=0.1.45->ultralytics) (0.16.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.13/dist-packages (from sympy>=1.13.3->torch>=1.8.0->ultralytics) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.13/dist-packages (from jinja2->torch>=1.8.0->ultralytics) (3.0.3)
Image loaded successfully!

Input Image



0: 640x640 4 persons, 11 cars, 2 buss, 6 traffic lights, 2 umbrellas, 222.7ms
 Speed: 11.2ms preprocess, 222.7ms inference, 1.4ms postprocess per image at shape (1, 3, 640, 640)

Object Detection Result



Detected Objects:

```
-----  
Object: car | Confidence: 0.88  
Object: car | Confidence: 0.77  
Object: bus | Confidence: 0.75  
Object: car | Confidence: 0.75  
Object: car | Confidence: 0.71  
Object: car | Confidence: 0.68  
Object: traffic light | Confidence: 0.66  
Object: traffic light | Confidence: 0.62  
Object: car | Confidence: 0.59  
Object: traffic light | Confidence: 0.47  
Object: person | Confidence: 0.47  
Object: car | Confidence: 0.46  
Object: umbrella | Confidence: 0.41  
Object: car | Confidence: 0.41  
Object: person | Confidence: 0.41  
Object: car | Confidence: 0.41  
Object: person | Confidence: 0.4  
Object: person | Confidence: 0.38  
Object: car | Confidence: 0.35  
Object: umbrella | Confidence: 0.35  
Object: traffic light | Confidence: 0.34  
Object: bus | Confidence: 0.29  
Object: car | Confidence: 0.27  
Object: traffic light | Confidence: 0.27  
Object: traffic light | Confidence: 0.26
```

Inference Time: 0.343 seconds

Model Used: YOLOv8 Nano

Model File: yolov8n.pt

Real-World Applications:

1. Autonomous Driving
2. Surveillance
3. Retail Analytics
4. Industrial Inspection

Conclusion:

YOLOv8 successfully detects and localizes objects using bounding boxes, class labels and confidence scores. The model is fast and suitable for real-time object detection.

Step 9: Suitability for Real-World Applications

The selected YOLOv8 object detection model is suitable for many real-world applications because it can detect and locate multiple objects quickly.

- Autonomous Driving: Detects cars, pedestrians, traffic signs, and other vehicles.
- Surveillance: Detects people and vehicles in security camera footage.
- Retail Analytics: Detects products and monitors customer activity.
- Industrial Inspection: Detects defective or damaged products.
- Traffic Monitoring: Detects and counts vehicles on roads.

Step 10: Advantages and Limitations

Advantages:

1. Fast and efficient object detection.
2. Can detect multiple objects in a single image.
3. Provides bounding boxes, class labels, and confidence scores.
4. Suitable for real-time applications.
5. Pre-trained models are easily available.
6. Can be trained for specific object detection tasks.

Limitations:

1. May have difficulty detecting very small objects.
2. Detection accuracy can decrease in poor lighting.
3. Objects hidden behind other objects may be missed.
4. Larger models require more computational resources.
5. Accuracy depends on the quality of the training dataset.

1. Differentiate between Image Classification, Object Detection, and Image Segmentation.

Technique	Meaning	Output
Image Classification	Identifies what object/category is present in an image.	Class label
Object Detection	Identifies objects and finds their locations.	Class label + Bounding box
Image Segmentation	Identifies objects at pixel level.	Pixel-wise mask

Example: For an image containing a dog:

- Classification → Dog
- Detection → Dog + bounding box
- Segmentation → Dog + exact pixels belonging to the dog

2. Explain the working principle of YOLO.

YOLO (You Only Look Once) is a real-time object detection algorithm.

Working:

1. The input image is given to the YOLO model.
2. The image is processed by a deep neural network.
3. YOLO predicts bounding boxes around objects.
4. It predicts the class of each object.
5. It also calculates a confidence score for each prediction.
6. Non-Maximum Suppression (NMS) is used to remove duplicate overlapping boxes.
7. The final detected objects are displayed with their labels and bounding boxes.

3. Compare YOLO, SSD, and Faster R-CNN.

Feature	YOLO	SSD	Faster R-CNN
Speed	Very fast	Fast	Slower
Accuracy	High	High	Generally very high
Real-time use	Excellent	Good	More difficult
Architecture	Single-stage	Single-stage	Two-stage
Applications	Surveillance, autonomous systems	Mobile/real-time detection	High-accuracy detection, research
Computation	Relatively lower	Moderate	Higher

4. What is a Bounding Box? Why is it important?

A bounding box is a rectangular box drawn around a detected object in an image.

It is generally represented using coordinates such as:

(x, y, width, height)

Importance:

- Shows the location of an object.
- Helps distinguish different objects.
- Allows the system to identify the object's position. Used to calculate metrics such as IoU.

Example: A box around a car tells the system where the car is located in the image.

5. Explain Confidence Score and IoU.

Confidence Score : A confidence score indicates how confident the model is about its prediction.

Example:

Person → 0.95

Car → 0.87

Dog → 0.72

A score of 0.95 means the model has high confidence in that detection.

IoU (Intersection over Union) : IoU measures how much the predicted bounding box overlaps with the actual/ground-truth bounding box.

Formula:

$$IoU = \frac{\text{Area of Intersection}}{\text{Area of Union}}$$

IoU ranges from 0 to 1.

0 → No overlap

1 → Perfect overlap

6. Advantages of Pre-trained Deep Learning Models over Traditional Image Processing

Pre-trained models have several advantages:

1. They are already trained on large datasets.
2. They require less training time and data for many tasks.
3. They can recognize complex patterns automatically.
4. They generally perform better on challenging images.
5. They can be fine-tuned for specific applications.
6. They reduce the need for manually designed features.

Example: YOLO trained on a large dataset can detect common objects without manually designing features such as edges and shapes.

7. Why are MS COCO and Pascal VOC widely used?

MS COCO and Pascal VOC are popular object detection datasets because they provide:

- Large collections of labeled images.
- Multiple object categories.
- Bounding box annotations.
- Standard evaluation metrics.

Common benchmarks for comparing different detection algorithms.

8. Five Real-World Applications of Object Detection

Five important applications are:

1. Autonomous Vehicles – Detect cars, pedestrians, traffic signs, etc.
2. Surveillance Systems – Detect people and vehicles.
3. Medical Imaging – Detect certain abnormalities or regions of interest.
4. Retail Analytics – Detect products and monitor shelves.
5. Industrial Inspection – Detect defective or damaged products.

9. Challenges in Object Detection

Object detection becomes difficult in complex real-world environments because of:

1. Poor lighting – Objects may be difficult to recognize.
2. Occlusion – Objects may be partially hidden.
3. Small objects – Very small objects are difficult to detect.
4. Background clutter – Complex backgrounds can confuse the model.
5. Different viewpoints – Objects can look different from different angles.
6. Object scale variation – Objects may appear very large or very small.
7. Similar-looking objects – Different classes may have similar appearances.
8. Motion blur – Moving objects may appear blurred.

10. How can Object Detection Systems be Improved?

Object detection can be improved using recent developments in deep learning and computer vision:

1. Larger and better-quality datasets can improve training.
2. Data augmentation can create more varied training examples.
3. Transfer learning can improve performance with less training data.
4. Advanced architectures such as newer YOLO versions and transformer-based detectors can improve detection.
5. Better feature extraction can improve detection of small and difficult objects.
6. Attention mechanisms can help the model focus on important regions.
7. Model optimization and quantization can make models faster and more efficient.
8. Better loss functions and training techniques can improve localization and classification accuracy.